Constraints in MySQL

What are Constraints?

Constraints are rules applied to table columns in MySQL to ensure that only **valid and accurate data** is stored in the database.

In simple words:

> **Constraints = Rules that control the data entered into a table.**

Why do we use constraints?

Constraints are used to:

- Maintain data integrity
- Prevent invalid data entry
- Avoid duplicate records
- Ensure relationships between tables
- Make the database reliable and consistent

integrity constraints:-

- 1) NOT NULL
- 2) UNIQUE
- 3) PRIMARY KEY

. 4) composite key

- 5) FOREIGN KEY or REFERENCECIAL INTEGRITY
- 6)CHECK

7) default constraint

Integrity Constraints in MySQL

Integrity constraints are rules that ensure the accuracy, consistency, and reliability of data stored in a database.

---

1) NOT NULL Constraint

Definition

The NOT NULL constraint ensures that a column cannot contain NULL (empty) values. Every row must have a value for that column.

Example

```
CREATE TABLE employees (
    emp_id INT,
    emp_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2)
);
```

Insert Valid Record

```
INSERT INTO employees VALUES
(101, 'Ravi', 35000);
```

Insert Invalid Record

```
INSERT INTO employees VALUES
(102, NULL, 40000);
```

Output

ERROR:

Column 'emp_name' cannot be null

---

2) UNIQUE Constraint

Definition

The UNIQUE constraint ensures that all values in a column are different. Duplicate values are not allowed.

Example

```
CREATE TABLE students (
    student_id INT,
    email VARCHAR(100) UNIQUE,
    name VARCHAR(50)
);
```

Insert Records

```
INSERT INTO students VALUES
(1, 'ravi@gmail.com', 'Ravi');

INSERT INTO students VALUES
(2, 'priya@gmail.com', 'Priya');
```

Duplicate Value

```
INSERT INTO students VALUES
(3, 'ravi@gmail.com', 'Mahesh');
```

Output

ERROR:

Duplicate entry 'ravi@gmail.com' for key

---

3) PRIMARY KEY Constraint

Definition

A PRIMARY KEY uniquely identifies every row in a table.

Rules

- Cannot contain NULL values.
- Cannot contain duplicate values.
- Only one Primary Key per table.

Example

```
CREATE TABLE products (

   product_id INT PRIMARY KEY,

   product_name VARCHAR(50),

   price DECIMAL(10,2)

);
```

Insert Records

```
INSERT INTO products VALUES

(101, 'Laptop', 55000);


INSERT INTO products VALUES

(102, 'Mouse', 800);
```

Duplicate Primary Key

```
INSERT INTO products VALUES

(101, 'Keyboard', 1200);
```

Output

ERROR:

Duplicate entry '101' for key 'PRIMARY'

---

4) Composite Key

Definition

A Composite Key is a Primary Key formed using two or more columns together.

Example

```
CREATE TABLE enrollment (
   student_id INT,
   course_id INT,
   enroll_date DATE,
   PRIMARY KEY(student_id, course_id)
);
```

Insert Records

```
INSERT INTO enrollment VALUES
(101, 1, '2026-07-21');

INSERT INTO enrollment VALUES
(101, 2, '2026-07-21');

INSERT INTO enrollment VALUES
(102, 1, '2026-07-21');
```

Duplicate Composite Key

```
INSERT INTO enrollment VALUES
```

(101, 1, '2026-07-22');

Output

ERROR:

Duplicate entry '101-1' for key 'PRIMARY'

---

5) FOREIGN KEY (Referential Integrity)

Definition

A Foreign Key creates a relationship between two tables. It ensures that a value in the child table must already exist in the parent table.

Parent Table

```
CREATE TABLE department (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);
```

Insert Records

```
INSERT INTO department VALUES
(101,'HR'),
(102,'IT'),
(103,'Finance');
```

Child Table

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY(dept_id)
    REFERENCES department(dept_id)
);
```

Valid Insert

```
INSERT INTO employee VALUES
(1,'Ravi',101);

INSERT INTO employee VALUES
(2,'Priya',102);
```

Invalid Insert

```
INSERT INTO employee VALUES
(3,'Mahesh',110);
```

Output

ERROR:

Cannot add or update a child row:

a foreign key constraint fails

---

6) CHECK Constraint

Definition

The CHECK constraint ensures that values satisfy a specified condition before insertion or update.

Example

```
CREATE TABLE exams (
   student_id INT,
   marks INT CHECK (marks BETWEEN 0 AND 100)
);
```

Valid Records

```
INSERT INTO exams VALUES
(101,85);

INSERT INTO exams VALUES
(102,65);
```

Invalid Record

```
INSERT INTO exams VALUES
(103,120);
```

Output

ERROR:

Check constraint violated

---

7) DEFAULT Constraint

Definition

The DEFAULT constraint automatically assigns a predefined value if no value is provided during insertion.

Example

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50) DEFAULT 'Hyderabad'
);
```

Insert Without City

```
INSERT INTO customers(customer_id, customer_name)
VALUES
(101,'Ravi');
```

View Table

```
SELECT * FROM customers;
```

Output

customer_id	customer_name	city
101	Ravi	Hyderabad

Insert With City

```
INSERT INTO customers VALUES
(102,'Priya','Bangalore');
```

Output

customer_id	customer_name	city
101	Ravi	Hyderabad
102	Priya	Bangalore

Summary Table

Constraint	Purpose	Duplicate Allowed	NULL Allowed
NOT NULL	Prevents NULL values	Yes	No
UNIQUE	Prevents duplicate values	No	Yes (one or more NULLs in MySQL, depending on version/usage)
PRIMARY KEY	Uniquely identifies each row	No	No
Composite Key	Combination of columns uniquely identifies a row	No (for the combination)	No (as part of the primary key)
FOREIGN KEY	Maintains parent-child relationship	Yes	Yes (unless also declared NOT NULL)

CHECK	Restricts values based on a condition	Depends on the condition	Depends on the column definition
DEFAULT	Assigns a default value when none is provided	Depends on the column	Depends on the column defin

What are Referential Actions?

Referential actions define what happens to child table data when the parent table data is updated or deleted.

They are used with FOREIGN KEY constraints.

1. ON DELETE CASCADE

Concept

When a parent row is deleted, all matching child rows are automatically deleted.

🔧 **What it does**

Keeps data consistent by removing dependent records automatically.

Example

```
-- Parent Table
CREATE TABLE Departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);
```

-- Child Table

```
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(100),
  dept_id INT,
  FOREIGN KEY (dept_id)
  REFERENCES Departments(dept_id)
  ON DELETE CASCADE
);
```

Insert Records

```
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR');

INSERT INTO Employees VALUES
(101, 'Rahul', 1),
(102, 'Sneha', 1),
(103, 'Amit', 2);
```

Operation

```
DELETE FROM Departments WHERE dept_id = 1;
```

Result

- **Dept 1 deleted**
- **Employees with dept_id = 1 also deleted automatically**

🔹 **2. ON UPDATE CASCADE**

Concept

When a parent key is updated, all matching child foreign keys are updated automatically.

What it does

Maintains consistency when primary key changes.

Example

```
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(100),
  dept_id INT,
  FOREIGN KEY (dept_id)
  REFERENCES Departments(dept_id)
  ON UPDATE CASCADE
);
```

Insert Records

```
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR');

INSERT INTO Employees VALUES
(101, 'Rahul', 1),
(102, 'Sneha', 1),
(103, 'Amit', 2);
```

Operation

```
UPDATE Departments SET dept_id = 10 WHERE dept_id = 1;
```

Result

- **dept_id changed from 1 → 10**
- **Employees dept_id also updated automatically to 10**

---

3. ON DELETE SET NULL

Concept

When a parent row is deleted, the child foreign key becomes NULL.

What it does

Keeps the child record but removes the relationship.

Column must allow NULL

Example

```
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(100),
  dept_id INT NULL,
  FOREIGN KEY (dept_id)
  REFERENCES Departments(dept_id)
  ON DELETE SET NULL
);
```

Insert Records

```
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR');
```

INSERT INTO Employees VALUES

(101, 'Rahul', 1),

(102, 'Sneha', 1),

(103, 'Amit', 2);

Operation

DELETE FROM Departments WHERE dept_id = 1;

Result

- **Dept 1 deleted**
- **Employees remain, but dept_id becomes NULL**

---

4. ON UPDATE SET NULL

Concept

When a parent key is updated, the child foreign key becomes NULL.

What it does

Breaks relationships instead of updating value.

Column must allow NULL

Example

CREATE TABLE Employees (

emp_id INT PRIMARY KEY,

name VARCHAR(100),

dept_id INT NULL,

FOREIGN KEY (dept_id)

REFERENCES Departments(dept_id)

ON UPDATE SET NULL

);

Insert Records

INSERT INTO Departments VALUES

(1, 'IT'),

(2, 'HR');

INSERT INTO Employees VALUES

(101, 'Rahul', 1),

(102, 'Sneha', 1),

(103, 'Amit', 2);

Operation

UPDATE Departments SET dept_id = 10 WHERE dept_id = 1;

Result

- **Parent updated**
- **Child dept_id becomes NULL (not 10)**

Final Summary

Action	When Parent Changes	Child Table Result
ON DELETE CASCADE	**Delete parent**	**Child rows deleted**
ON UPDATE CASCADE	**Update parent**	**Child updated**

ON DELETE SET NULL	**Delete parent**	**Child set to NULL**
ON UPDATE SET NULL	**Update parent**	**Child set to NULL**